

FLARQ Hackathon Compliance Gap Analysis Report

Google Cloud Rapid Agent Hackathon - MongoDB Track

Assessment Date: May 17, 2026 | Version 2.0

Executive Summary

This report evaluates the FLARQ project (flarq.ai) against every requirement of the Google Cloud Rapid Agent Hackathon. After a complete re-analysis of the codebase following recent changes, the project demonstrates strong engineering fundamentals with a genuine MCP integration and a well-structured frontend. However, it has **one critical compliance gap** that poses a disqualification risk: the project does not use **Google Cloud Agent Builder**, the centerpiece technology of this hackathon. Additionally, the project lacks a production deployment on Cloud Run, which is a submission requirement. Without addressing these two issues, the project cannot be competitively submitted.

Assessment	Status	Risk Level
Overall Hackathon Compliance	NOT READY TO SUBMIT	CRITICAL
Google Cloud Agent Builder Usage	NON-COMPLIANT	DISQUALIFICATION
MCP + MongoDB Integration	COMPLIANT	LOW
Production Deployment (Cloud Run)	NON-COMPLIANT	HIGH
Open Source License	PARTIAL	MEDIUM
Design & UX Quality	GOOD	LOW

Judging Criteria Estimated Scores

Criterion	Weight	Score	Key Gap
Technological Implementation	25%	5/10	No Agent Builder, no Cloud Run, no Secret Manager
Design	25%	7/10	Agent page is basic chatbot, no streaming
Potential Impact	25%	7/10	Crowded market, individual-level impact

Quality of the Idea	25%	6/10	Not novel; Vector Search is stubbed
---------------------	-----	------	-------------------------------------

Weighted Overall Score: 6.25 / 10 (but Agent Builder gap likely overrides scores)

1. Hackathon Requirements Compliance Matrix

The hackathon states: "Build a functional agent - powered by Gemini and Google Cloud Agent Builder - that integrates a Partner Entity's MCP server to solve a real challenge." Below is a detailed assessment of FLARQ's compliance against each explicit requirement from the problem statement.

Requirement	Status	Evidence / Gap
Powered by Gemini	COMPLIANT	Uses vertexai.generative_models.GenerativeModel (gemini-2.0-flash-001) via Vertex AI SDK. 6 function declarations for tool use. Retry logic with exponential backoff.
Powered by Google Cloud Agent Builder	NON-COMPLIANT	AGENT_BUILDER_ID field exists in config.py but is NEVER referenced anywhere in codebase. The agent loop is a custom implementation using raw GenerativeModel.generate_with_tools(). No google-cloud-discoveryengine or Agent Platform SDK in requirements.txt.
Move Beyond Chat (use tools)	COMPLIANT	Agent uses 6 tools: get_profile_summary, search_applications, get_analytics_insight, get_stale_applications, generate_follow_up, update_application_status. These accomplish real tasks, not just Q&A.;
Multi-Step Mission (plan + execute)	PARTIAL	Agent loop runs max 8 iterations with tool chaining. Gemini can chain calls (e.g., find stale apps then generate follow-up). However, there is no explicit planning step shown to the user, and no human-in-the-loop confirmation before actions.
Partner MCP Integration (MongoDB)	COMPLIANT	Genuine MCP server with 9 MongoDB tools via proper MCP SDK (stdio transport). All data operations flow through MCP: analytics aggregations, profile lookups, application CRUD, conversation storage. Collection allowlist prevents arbitrary access.
Hosted Project URL	NON-COMPLIANT	No cloudbuild.yaml, no Cloud Run deployment, no CI/CD pipeline. Only docker-compose.yml for local development. No evidence of any cloud deployment.
Public Repo with License	PARTIAL	MIT License exists at repo root (good). But copyright year says 2026 (likely typo). Also, README.md has zero setup instructions and does not mention Agent Builder.

Demo Video (~3 min)	PARTIAL	No video found in repo (expected - typically uploaded to YouTube/Devpost separately). Cannot verify. Must be created before submission.
---------------------	---------	---

2. Critical Gap: Google Cloud Agent Builder (Disqualification Risk)

This is the single most important finding in this entire audit. The hackathon's core requirement explicitly states: "Build a functional agent - powered by Gemini and **Google Cloud Agent Builder**." The current FLARQ implementation uses the raw Vertex AI SDK directly, bypassing Agent Builder entirely. This is not a minor gap; it is a fundamental architectural mismatch with the hackathon's primary technology requirement. Judges will immediately notice this absence, and it could result in disqualification regardless of the project's other merits.

2.1 Current Architecture vs. Required Architecture

Aspect	Current Implementation	Required by Hackathon
Agent Framework	Custom hand-rolled ReAct loop in agent_builder.py (max 8 iterations)	Google Cloud Agent Builder (managed orchestration, grounding, tool use)
Gemini Integration	Direct vertexai.generative_models.GenerativeModel + generate_with_tools()	Agent Builder SDK with extensions, data stores, and deployment
Tool Registration	Manual FunctionDeclaration objects in vertex_client.py	Agent Builder Extensions (pre-built or custom API extensions)
Knowledge/Grrounding	None - no data store, no grounding	Agent Builder Data Stores (index PDFs, websites, BigQuery)
Deployment	Docker + FastAPI (manual)	Agent Builder Deployment (web interface + API) or Cloud Run
Safety	None - no SafetySettings configured	Gemini Enterprise Agent Platform Safety Settings
State/Secrets	.env file (plaintext)	Google Secret Manager

2.2 Dead Config Proof

The config.py file defines an agent_builder_id field (line 28), and .env.example includes AGENT_BUILDER_ID= (empty). However, a search of the entire codebase reveals zero references to settings.agent_builder_id anywhere in the business logic. This field was likely added to "check the box" but was never wired to the Agent Builder API. The requirements.txt also has no google-cloud-discoveryengine

or google-cloud-agentplatform package. This is the strongest evidence that Agent Builder was planned but never implemented.

2.3 Phase 1-5 Technology Compliance

The hackathon outlines a 5-phase technical journey with specific Google Cloud technologies. Here is FLARQ's coverage across all recommended phases:

Phase	Technology	Used?
Phase 1	Gemini Enterprise Agent Platform API	PARTIAL
Phase 1	Agent Builder Guide (low-code path)	NON-COMPLIANT
Phase 1	Agent Platform SDK for Python	NON-COMPLIANT
Phase 2	Agent Builder Extensions (tool use)	PARTIAL
Phase 2	Agent Builder Data Stores (grounding)	NON-COMPLIANT
Phase 3	MongoDB MCP Integration	COMPLIANT
Phase 4	Agent Runtime (orchestration)	NON-COMPLIANT
Phase 4	Secret Manager	NON-COMPLIANT
Phase 5	Cloud Run Deployment	NON-COMPLIANT
Phase 5	Safety Settings	NON-COMPLIANT

Coverage: 1.5 / 10 technologies (15%) - Only MongoDB MCP is fully compliant.

3. MCP Integration Assessment (MongoDB Track)

The MCP integration is the project's strongest point and the primary reason it can compete in the MongoDB track. FLARQ implements a genuine, custom MCP server with proper protocol compliance, not a thin wrapper around Motor. All data operations (except auth) flow through the MCP layer, giving the agent real superpowers over the user's data.

3.1 MCP Protocol Compliance

The MCP server uses the official Python MCP SDK (mcp >= 1.0.0 in requirements.txt) with proper @server.list_tools() and @server.call_tool() decorators. The client uses ClientSession with stdio transport (StdioServerParameters), spawning the server as a subprocess. This is a real MCP implementation, not a REST API wrapper. The server exposes 9 tools covering the full CRUD spectrum plus aggregation and vector search.

3.2 MongoDB Feature Depth

Feature	Status	Details
Aggregation Pipeline	COMPLIANT	4 dedicated modules with \$lookup, \$unwind, \$group, \$addFields, \$convert, \$dateToString. Cross-collection joins between applications and job_descriptions.
Atlas Vector Search	PARTIAL	\$vectorSearch pipeline exists in MCP tool but catches all exceptions and returns []. No embedding generation code exists. This is dead/stub code.
Atlas Text Search	PARTIAL	One text index on requirements field. No \$search queries - uses \$regex instead. Should upgrade to Atlas Search for production.
Change Streams	NON-COMPLIANT	Not implemented. Would add real-time notifications for application status changes.
Schema Introspection	COMPLIANT	mongodb_inspect_schema tool samples documents, extracts field names, shows indexes. Unique feature.
Soft Delete Pattern	COMPLIANT	All deletes set deleted: true + deleted_at timestamp. No hard deletes anywhere.
Collection Allowlist	COMPLIANT	ALLOWED_COLLECTIONS frozenset prevents arbitrary collection access. Security best practice.

3.3 MCP Integration Weakness: Auth Bypasses MCP

There is a dual data access pattern in the codebase. Auth endpoints use UserRepository with direct Motor (AsyncIOMotorDatabase) access, bypassing the MCP layer entirely. This undermines the "all data flows through MCP" narrative that would score highly with MongoDB judges. For maximum hackathon impact, even authentication should route through the MCP client.

4. Frontend and Design Assessment

The frontend is built with React 19, TypeScript, Tailwind CSS, Framer Motion, Zustand, and TanStack Query. The overall design quality is good (7/10), with a cohesive teal-cyan color system and consistent component library. However, there are significant issues that would lower the "Design" judging score, particularly around the Agent page.

4.1 Page-by-Page Design Scores

Page	Score	Key Issues
Landing Page	7/10	Dead "See how it works" link, no mobile hero visual, fabricated 10,000+ stat
Auth Page	8/10	No Google OAuth button (critical for Google hackathon), no forgot password flow

Dashboard	7/10	CSS hover bug (invalid group-hover:teal-cta), Sidebar component never rendered, no charts
Profile	6/10	Dead skill remove buttons, 3 unused component files, no education section displayed
Analyze	8/10	Best-designed page. No edit after submission, no save-as-draft
Applications	8/10	Kanban breaks on mobile, window.confirm for delete, no edit modal
Analytics	6/10	3 unused chart components, weakly typed data, hardcoded "30 days"
Agent Page	5/10	BASIC CHATBOT, NOT AN AGENT. No streaming, no thinking steps, no rich responses, no markdown rendering

4.2 Agent Page: The Critical Weakness

The Agent page is the hackathon's centerpiece feature, yet it currently feels like a basic chatbot rather than a powerful AI agent. This is the single biggest design gap because the hackathon explicitly requires agents that "accomplish tasks" and "handle complex goals." When a user asks the agent to "find stale applications and draft follow-ups," the experience should feel like watching an intelligent assistant work through the problem step by step - not like waiting 10 seconds for a chatbot to dump a text response.

Feature	Status	Impact on Judges
Streaming responses	NON-COMPLIANT	CRITICAL - 10s silence then dump feels laggy and unagent-like
Real-time tool execution timeline	NON-COMPLIANT	HIGH - chips appear after response, not during execution
Thinking/reasoning visualization	NON-COMPLIANT	HIGH - no "Analyzing your data..." step before tool calls
Markdown rendering	NON-COMPLIANT	HIGH - plain text responses look primitive
Rich inline cards	NON-COMPLIANT	MEDIUM - structured data shown as text, not interactive cards
Cancel/abort button	NON-COMPLIANT	LOW - code creates AbortController but no UI button
Conversation management	PARTIAL	LOW - sidebar works but no delete/rename

4.3 Dead Code and Performance Issues

The frontend contains 12 dead component/store files (~600 lines of unused code) that add bundle bloat and code confusion: ResumeUpload.tsx, ProfileCard.tsx, SkillTags.tsx, ResponseRateChart.tsx, PatternSummary.tsx, InsightCard.tsx, JDPasteBox.tsx, GapAnalysisCard.tsx, CoverLetterModal.tsx, Sidebar.tsx, profileStore.ts, applicationStore.ts. Additionally, there is no code splitting - all pages are

eagerly imported, meaning Recharts (~450KB), Framer Motion (~150KB), and DnD Kit are loaded even on pages that do not use them. Lazy loading via `React.lazy()` and `Suspense` should be implemented for every page.

4.4 Missing Google OAuth Button

This is a Google Cloud hackathon, and the auth page has no "Sign in with Google" button. This is a significant oversight that judges will notice immediately. Adding a Google OAuth button using `@react-oauth/google` would be a strong signal of Google Cloud integration and would improve both the Design and Technological Implementation scores.

5. Submission Readiness Checklist

The hackathon requires specific deliverables for submission. Below is the current status of each requirement, along with what needs to be done before the project can be submitted.

Deliverable	Status	Action Required
Hosted Project URL	NON-COMPLIANT	Create <code>cloudbuild.yaml</code> , deploy backend + frontend to Cloud Run, configure environment variables and secrets. Estimated: 2-3 hours.
Public GitHub Repository	PARTIAL	Repo exists but README has no setup instructions, no mention of Agent Builder, and no architecture diagram. Must improve before judges review code.
Open Source License (visible)	PARTIAL	MIT License exists at root. Fix copyright year from 2026 to 2025. Ensure it is visible in GitHub About section.
Demo Video (~3 min)	NON-COMPLIANT	Must create a 3-minute demo video showing: problem statement, agent capabilities, MCP tool usage, and Google Cloud integration. Upload to YouTube, link on Devpost.
Track Selection (MongoDB)	COMPLIANT	MCP integration with MongoDB is genuine and deep. MongoDB track is the correct choice.
Devpost Submission Form	NON-COMPLIANT	Must complete the Devpost form with project description, tech stack, what you built, how you built it, and challenges.

6. Priority Fix List (Ordered by Impact)

Below are the specific fixes needed, ordered by their impact on hackathon qualification and scoring. Items marked "DISQUALIFICATION RISK" must be addressed before submission; the project cannot be competitively submitted without them.

#	Fix	Risk	Est. Time	Details
1	Integrate Google Cloud Agent Builder	DISQUALIFY	3-4 hrs	Add google-cloud-discoveryengine to requirements. Create AgentBuilderClient class. Wire AGENT_BUILDER_ID config. Minimum: Agent Builder as first-pass routing, fall back to custom Vertex AI agent for complex tool calls.
2	Deploy to Cloud Run	DISQUALIFY	2-3 hrs	Create cloudbuild.yaml. Create frontend/Dockerfile (nginx). Create nginx.conf. Deploy both services. Configure secrets. Get hosted URL.
3	Add Secret Manager	HIGH	1 hr	Add google-cloud-secret-manager to requirements. Create secrets.py module. Move MONGODB_URI and JWT_SECRET_KEY to Secret Manager. Update config.py to load from secrets.
4	Add Safety Settings to Gemini	MEDIUM	30 min	Import SafetySetting, HarmCategory, HarmBlockThreshold from vertexai. Add DEFAULT_SAFETY_SETTINGS to all Gemini calls. Document in README.
5	Upgrade Agent Page UX	HIGH	4-6 hrs	Add streaming (SSE). Show real-time tool execution timeline. Add markdown rendering (react-markdown). Add "thinking" indicator. Show progress per tool call. This is the #1 design improvement.
6	Add Google OAuth	MEDIUM	1-2 hrs	Add "Sign in with Google" button on auth page. Use @react-oauth/google. Configure OAuth consent screen in GCP. Critical signal for Google Cloud hackathon.
7	Activate Atlas Vector Search	MEDIUM	2-3 hrs	Generate embeddings using text-embedding-004. Create Atlas Vector Search index. Wire mongodb_vector_search MCP tool to return real results. This makes the MongoDB track competitive.
8	Fix README + License	LOW	30 min	Fix LICENSE year (2026 to 2025). Add setup instructions to README. Add architecture diagram. Mention Agent Builder, Gemini, MCP, Cloud Run. Make it judge-friendly.
9	Remove 12 dead frontend files	LOW	15 min	Delete: ResumeUpload.tsx, ProfileCard.tsx, SkillTags.tsx, ResponseRateChart.tsx, PatternSummary.tsx, InsightCard.tsx, JDPasteBox.tsx, GapAnalysisCard.tsx, CoverLetterModal.tsx, Sidebar.tsx, profileStore.ts, applicationStore.ts.
10	Add lazy loading (React.lazy)	LOW	30 min	Wrap every page in React.lazy() + Suspense. Critical for perceived performance in demos. Reduces initial bundle by ~600KB.

7. Backend Code Quality Assessment

While the primary focus of this audit is hackathon compliance, the backend code quality directly impacts the "Technological Implementation" judging criterion. Below are the key findings from the complete code review.

7.1 Strengths

- MCP protocol implementation is genuinely correct and deep (not a wrapper) - all data operations flow through real MCP tool calls with proper `@server.list_tools()` and `@server.call_tool()` decorators.
- Aggregation pipelines are sophisticated and production-quality, using `$lookup` with `$convert` for cross-collection joins, multi-stage `$group/$sort` pipelines, and `$dateToString` for time-based analytics.
- Multi-step agent loop with parallel tool execution via `asyncio.gather` - up to 8 iterations with conversation history windowing.
- Security: JWT + bcrypt + token rotation + prompt injection defense + collection allowlist in MCP server + rate limiting on all endpoints.
- Structured logging with request IDs, exponential backoff for Vertex AI retries (3 attempts), and JSON parse fallback for Gemini responses.
- Docker + docker-compose ready for local development with proper volume mounts and `env_file` support.

7.2 Issues

- No Agent Builder usage (CRITICAL - covered in Section 2). The hand-rolled agent loop is well-implemented but does not use the hackathon's required technology.
- Auth bypasses MCP layer - `UserRepository` uses direct Motor access instead of routing through the MCP client. This undermines the "all data through MCP" narrative.
- No Safety Settings on any Gemini call - the hackathon recommends configuring Gemini Enterprise Agent Platform Safety Settings, and their absence could be noticed by judges.
- MCP server spawns as a subprocess per container instance - the `stdio` transport works but is not ideal for Cloud Run. Consider SSE transport for production.
- No Cloud Run deployment configuration - no `cloudbuild.yaml`, no `service.yaml`, no Terraform. Only docker-compose for local dev.
- No Secret Manager integration - all secrets from `.env` file in plaintext. The hackathon recommends Google Secret Manager.
- Vector Search is scaffolded but non-functional - the `$vectorSearch` pipeline catches all exceptions and returns `[]`. No embedding generation code exists.

8. Recommended Action Plan

Based on the findings in this audit, here is the recommended action plan organized by priority and timeline. The total estimated effort is 15-20 hours of focused development.

8.1 Day 1: Critical Disqualification Fixes (6-7 hours)

Agent Builder Integration (3-4 hrs): Add google-cloud-discoveryengine to requirements.txt. Create an AgentBuilderClient class that uses the Discovery Engine ConversationalSearchService API. Wire the AGENT_BUILDER_ID config field. Minimum viable approach: Agent Builder as first-pass routing for simple queries, fall back to the custom Vertex AI agent for complex tool calls that require MCP interaction.

Cloud Run Deployment (2-3 hrs): Create cloudbuild.yaml with build steps for backend and frontend. Create frontend/Dockerfile (multi-stage: Node build + Nginx serve). Create nginx.conf with SPA routing. Deploy both services to Cloud Run. Configure MONGODB_URI and JWT_SECRET_KEY as environment variables initially (upgrade to Secret Manager on Day 2). Get the hosted URL for submission.

8.2 Day 2: High-Impact Score Improvements (5-7 hours)

Agent Page UX Overhaul (4-6 hrs): Add Server-Sent Events (SSE) for streaming responses. Implement real-time tool execution timeline showing each MCP tool call as it happens ("Searching your profile..." then "Running analytics..." then "Drafting follow-up..."). Add react-markdown + remark-gfm for rich response rendering. Add a "thinking" indicator before tool calls begin. These changes alone would bump the Agent page score from 5/10 to 8/10 and dramatically improve the demo experience.

Secret Manager Integration (1 hr): Add google-cloud-secret-manager to requirements. Create a secrets.py module that loads MONGODB_URI, JWT_SECRET_KEY, and other sensitive values from Google Secret Manager. Update config.py to use Secret Manager as the primary source with .env fallback for local development.

8.3 Day 3: Polish and Submission (4-6 hours)

Safety Settings (30 min): Add SafetySetting configurations for all four harm categories (harassment, hate speech, sexually explicit, dangerous content) at BLOCK_MEDIUM_AND_ABOVE threshold to every Gemini call.

Google OAuth (1-2 hrs): Add "Sign in with Google" button to the auth page. Configure OAuth consent screen in Google Cloud Console. Use @react-oauth/google or similar library. This is a critical signal for a Google Cloud hackathon.

Atlas Vector Search (2-3 hrs): Generate embeddings using text-embedding-004 model. Create Atlas Vector Search index on the embedding field. Wire the mongodb_vector_search MCP tool to return real semantic search results. This transforms the MongoDB integration from "good CRUD" to "intelligent search."

README and Submission Polish (1 hr): Fix LICENSE year. Add comprehensive README with setup instructions, architecture diagram, Agent Builder mention, MCP explanation, and screenshots. Create Devpost submission. Record 3-minute demo video.

Final Verdict

FLARQ is a well-engineered project with a genuine MCP + MongoDB integration and a clean frontend. The team has built something that genuinely solves a real-world problem. However, in its current state, it **cannot be competitively submitted** to the Google Cloud Rapid Agent Hackathon because it does not use the hackathon's core required technology (Agent Builder) and lacks a production deployment on Cloud Run. The good news is that both of these gaps are fixable within 6-7 hours of focused work. The MCP integration and MongoDB feature depth are genuinely competitive - once Agent Builder and Cloud Run are added, this project has strong potential to place in the MongoDB track.

Recommendation: Do not submit until Agent Builder integration and Cloud Run deployment are complete. These are not optional improvements - they are qualification requirements. Once fixed, focus the demo video on the agent's multi-step reasoning, real-time MCP tool execution, and the depth of the MongoDB integration (aggregation pipelines, vector search, schema introspection). This is what will differentiate FLARQ from other entries.